

--	--

NVEC - Na Volta Eu Compro

Documento de Arquitetura de Software

Versão 0.1

--	--

Histórico de Revisões			
Versão	Data	Descrição	Autor
0.1	05/05/2026	Primeira versão de validação	Gusttavo Castro, Felipe Matheus, Raphael Binatti Filardi

--	--

Sumário

1. INTRODUÇÃO	4
2. IDENTIFICAÇÃO DO PROJETO	4
3. REPRESENTAÇÃO ARQUITETURAL	4
4. METAS E RESTRIÇÕES DA ARQUITETURA	4
5. VISÃO DE CASOS DE USO	5
5.1. Realizações de Casos de Uso	5
6. VISÃO LÓGICA	5
6.1. Visão Geral	5
6.2. Pacotes de Design Significativos do Ponto de Vista da Arquitetura	5
6.3. Camadas	5
7. VISÃO DE PROCESSOS	6
8. VISÃO DE IMPLANTAÇÃO	6
9. VISÃO DA IMPLEMENTAÇÃO	6
9.1. Visão Geral	6
10. VISÃO DE DADOS (OPCIONAL)	6
11. TAMANHO E DESEMPENHO	7
12. QUALIDADE	7
13. ANEXOS	7
14. REFERÊNCIAS	7
15. APROVAÇÕES	7

--	--

Documento de Arquitetura de Software

1. Introdução

A gestão das finanças pessoais é um desafio que afeta significativamente a qualidade de vida de indivíduos e famílias em todo o mundo. O crescente endividamento, a falta de educação financeira e a dificuldade de encontrar produtos acessíveis são problemas que se tornaram cada vez mais comuns. Na região Metropolitana da Baixada Santista, essas questões não são exceção, e a necessidade de soluções eficazes é evidente.

Neste contexto, este projeto tem como objetivo apresentar um software que visa abordar esses desafios por meio do desenvolvimento de um aplicativo móvel dedicado à gestão financeira pessoal. O projeto tem como alvo a população da Baixada Santista, uma região caracterizada por sua atividade econômica intensa, mas também pelos desafios financeiros que seus residentes enfrentam devido ao alto custo de vida.

O aplicativo proposto oferecerá aos usuários ferramentas para registrar suas despesas, comparar preços de produtos, estabelecer limites de gastos e obter um maior controle sobre suas finanças. Além disso, fornecerá informações educacionais sobre gestão financeira, visando melhorar a conscientização financeira e promover hábitos de consumo mais saudáveis.

2. Identificação do Projeto

Projeto	<i>NVEC - Na Volta Eu Compro</i>
Requisitante	<i>População da Baixada Santista</i>
Gerente de Projetos	<i>Gusttavo Leandro de Castro Tavares</i>

3. Representação Arquitetural

O sistema Na Volta Eu Compro adota a arquitetura Ports and Adapters (também conhecida como Arquitetura Hexagonal), proposta por Alistair Cockburn. Nesse modelo, o núcleo da aplicação — composto pelas regras de negócio e pelos domínios do sistema — é completamente isolado de detalhes tecnológicos externos, como banco de dados, frameworks e interfaces de usuário. A comunicação entre o núcleo e o mundo externo ocorre exclusivamente por meio de portas (interfaces bem definidas) e adaptadores (implementações concretas dessas interfaces).

Essa escolha se justifica pela natureza multiplataforma do projeto: embora a primeira entrega seja um aplicativo móvel desenvolvido em React Native, a arquitetura deve suportar, sem reestruturação do domínio, a adição futura de um cliente web e de um cliente desktop. Os adaptadores de entrada serão substituídos ou

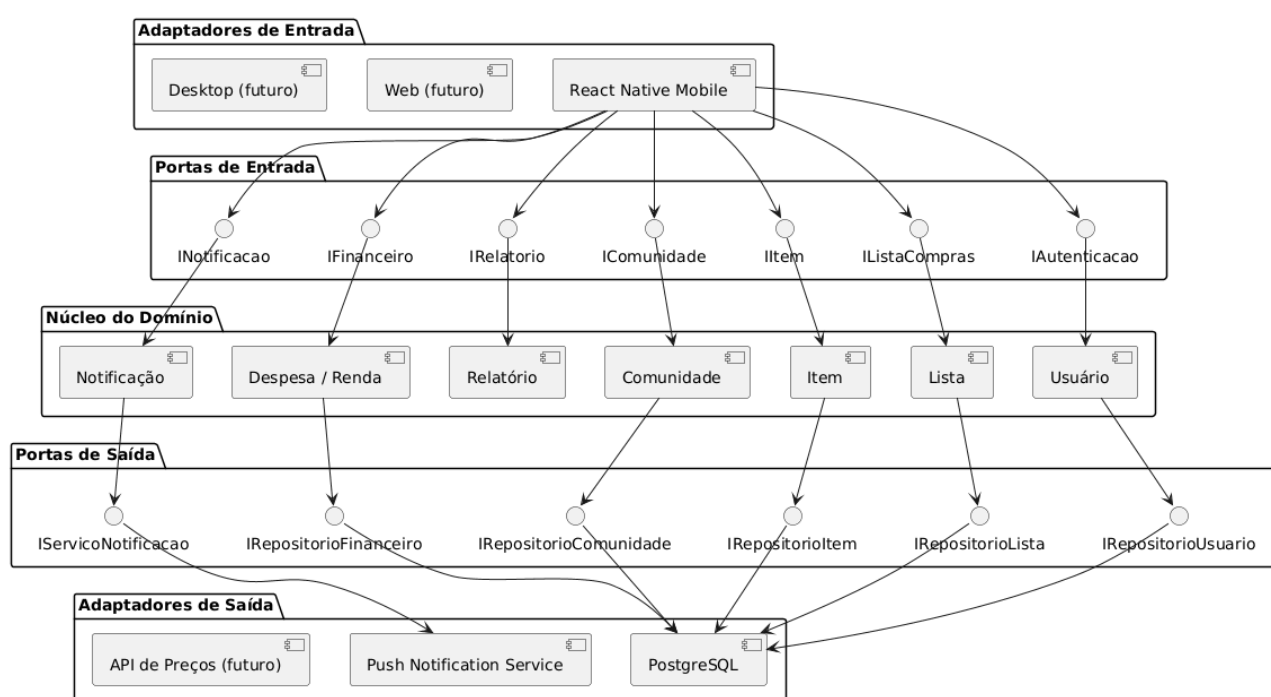
--	--

adicionados sem que o núcleo seja alterado. O mesmo princípio se aplica à camada de persistência: o adaptador PostgreSQL pode ser trocado ou complementado sem impacto nas regras de negócio.

A arquitetura é organizada nas seguintes visões, descritas nas seções seguintes:

- **Visão de Casos de Uso:** descreve as funcionalidades do sistema sob a perspectiva dos atores.
- **Visão Lógica:** descreve a organização do domínio em pacotes, classes e responsabilidades.
- **Visão de Processos:** descreve os fluxos de interação entre os componentes em tempo de execução, representados pelos diagramas de sequência.
- **Visão de Implementação:** descreve a organização física do código em camadas e componentes.
- **Visão de Implantação:** descreve como os componentes são distribuídos nos nós físicos e lógicos da infraestrutura.

A figura abaixo representa a estrutura geral da arquitetura adotada:



4. Metas e Restrições da Arquitetura

4.1. Realizações de Casos de Uso

MA01	Multiplataforma	A arquitetura deve permitir a adição de cliente web e desktop sem alteração do núcleo de domínio.
MA02	Isolamento do domínio	As regras de negócio não devem depender de framework, banco de dados ou bibliotecas externas.
MA03	Testabilidade	O núcleo deve ser testável de forma independente de qualquer adaptador.

--	--

MA04	Extensibilidade	Novos adaptadores de entrada e saída devem poder ser adicionados sem reestruturação arquitetural.
MA05	Segurança	Dados pessoais e financeiros dos usuários devem ser protegidos conforme a LGPD (Lei nº 13.709/2018).
MA06	Disponibilidade	O sistema deve estar disponível durante a maior parte do tempo, com mínima indisponibilidade para manutenção.
MA07	Usabilidade	A interface deve ser intuitiva para usuários de diferentes níveis de experiência tecnológica.

4.2. Restrições Arquiteturais

RA01	Linguagem do cliente	O cliente mobile é desenvolvido exclusivamente em React Native.
RA02	Linguagem do servidor	O backend é desenvolvido em Node.js.
RA03	Banco de dados	O sistema utiliza PostgreSQL como único adaptador de persistência na versão atual.
RA04	Notificações	O serviço de push é implementado via React Native Notifications
RA05	Acesso por faixa etária	O sistema não permite cadastro de usuários menores de 16 anos (RF01).
RA06	Conformidade legal	O sistema deve estar em conformidade com a LGPD e o Marco Cível da Internet (Lei nº 12.965/2014).
RA07	Modelo de assinatura	O sistema opera no modelo freemium: funcionalidades básicas gratuitas e funcionalidades avançadas mediante assinatura.
RA08	Limite de listas gratuitas	Usuários do plano gratuito podem criar no máximo 30 listas (RN03).

4.3. Estratégias de Design

A modelagem do sistema segue a notação UML 2.x, contemplando os seguintes diagramas:

- Diagrama de Casos de Uso
- Diagrama de Classes
- Diagrama de Sequência
- Diagrama de Pacotes
- Diagrama de Componentes

4.4. Ferramentas de Desenvolvimento

Cliente Mobile	React Native
-----------------------	--------------

--	--

Backend/API	Node.js
Banco de Dados	PostgreSQL
Notificações	React Native Push Notifications
Modelagem UML	PlantUML
Prototipação	Figma
Versionamento	Git/GitHub
Gerenciamento	Scrum

5. Visão de Casos de Uso

Esta seção apresenta os casos de uso que representam a funcionalidade central do sistema *Na Volta Eu Compro*. Os casos de uso selecionados cobrem os principais fluxos arquiteturais do sistema, atravessando as camadas de apresentação, domínio e persistência, e são referenciados pela numeração dos requisitos funcionais correspondentes.

1.1. Realizações de Casos de Uso

A tabela abaixo lista todos os casos de uso do sistema, com suas descrições e rastreabilidade com os requisitos funcionais:

UC01	Manter Usuário	Permitir incluir, alterar, excluir e pesquisar usuários no sistema.	RF01, RF02, RF03, RF04
UC02	Manter Lista de Compras	Permitir incluir, alterar, excluir e pesquisar listas de compras.	RF09, RF10, RF11, RF12
UC03	Manter Produto	Permite incluir, alterar, excluir e pesquisar produtos.	RF13, RF14, RF15, RF16
UC04	Gerenciar Quantidade de Produtos	Permite controlar a quantidade de produtos dentro de uma lista respeitando o limite de gastos.	RF17
UC05	Manter Comunidade	Permite incluir, alterar, excluir e pesquisar comunidades de lista.	RF18, RF19, RF20, RF21
UC06	Emitir Relatório de Gastos	Permite emitir relatório gráfico de gastos por período escolhido.	RF22
UC07	Emitir Relatório de Economias	Permite emitir relatório de economias por período escolhido.	RF23
UC08	Manter Limite de Gastos	Permite inserir, alterar e excluir o	RF24, RF25,

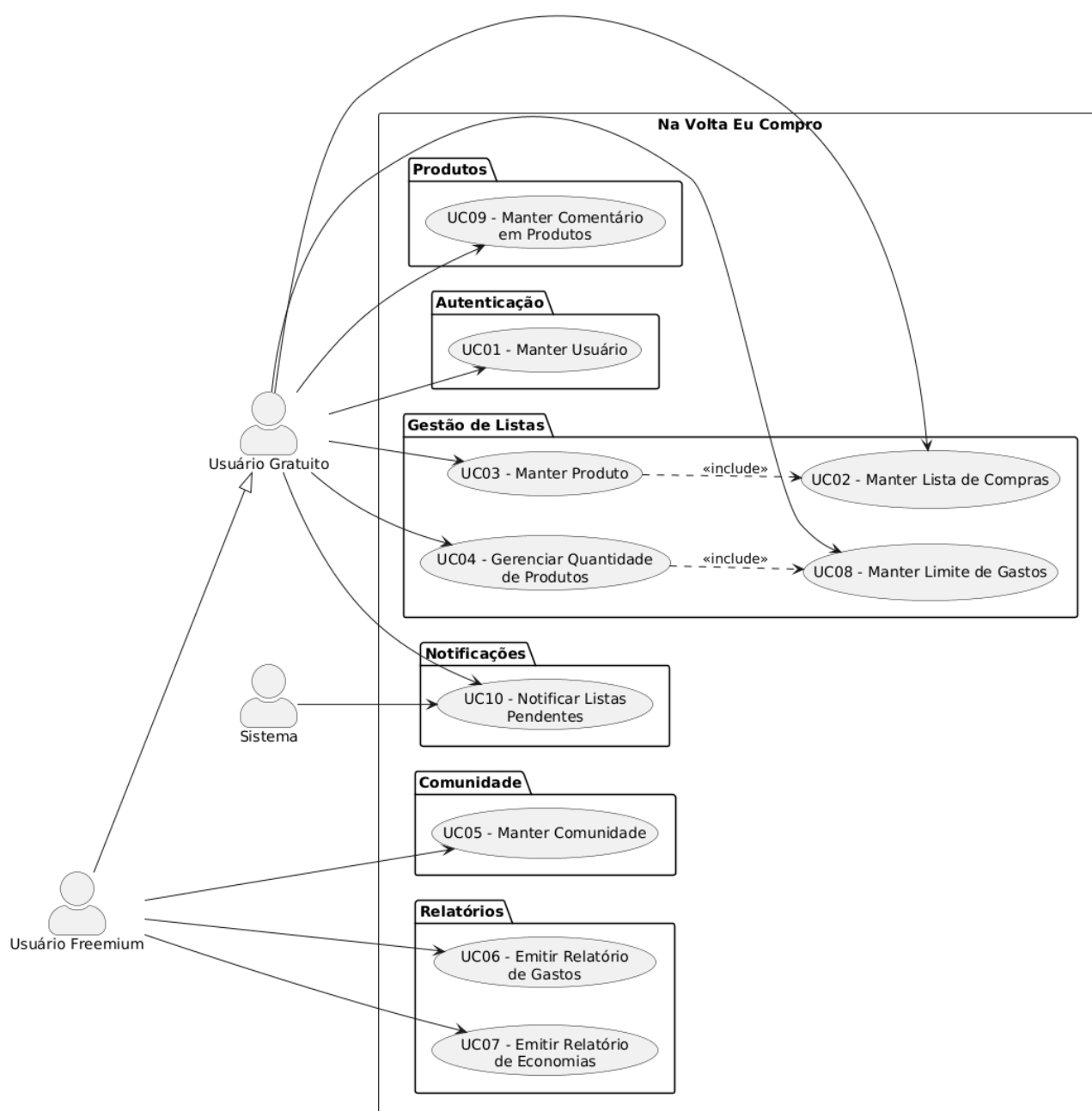
--	--

		limite de gastos de uma lista.	RF26
UC09	Manter Comentário em Produtos	Permite incluir, alterar e excluir comentários em produtos.	RF27, RF28, RF29

Os diagramas a seguir apresentam o modelo completo de casos de uso, organizados por atores.

Atores identificados:

- **Usuário Gratuito:** ator principal com acesso às funcionalidades básicas do sistema.
- **Usuários Premium:** extends Usuário Gratuito, com acesso a funcionalidades avançadas mediante assinatura.
- **Sistema:** ator secundário responsável por notificações e geração de relatórios automáticos.



--	--

6. Visão Lógica

Esta seção descreve a organização do modelo de design do sistema *Na Volta Eu Compro* em termos de pacotes, classes e responsabilidades, seguindo a arquitetura Ports and Adapters. O domínio é organizado em pacotes coesos, cada um representando um subdomínio funcional do sistema.

1.2. Visão Geral

O sistema é decomposto em cinco pacotes principais de domínio, mais os pacotes transversais de infraestrutura. A hierarquia reflete a separação entre núcleo de domínio, portas e adaptadores.



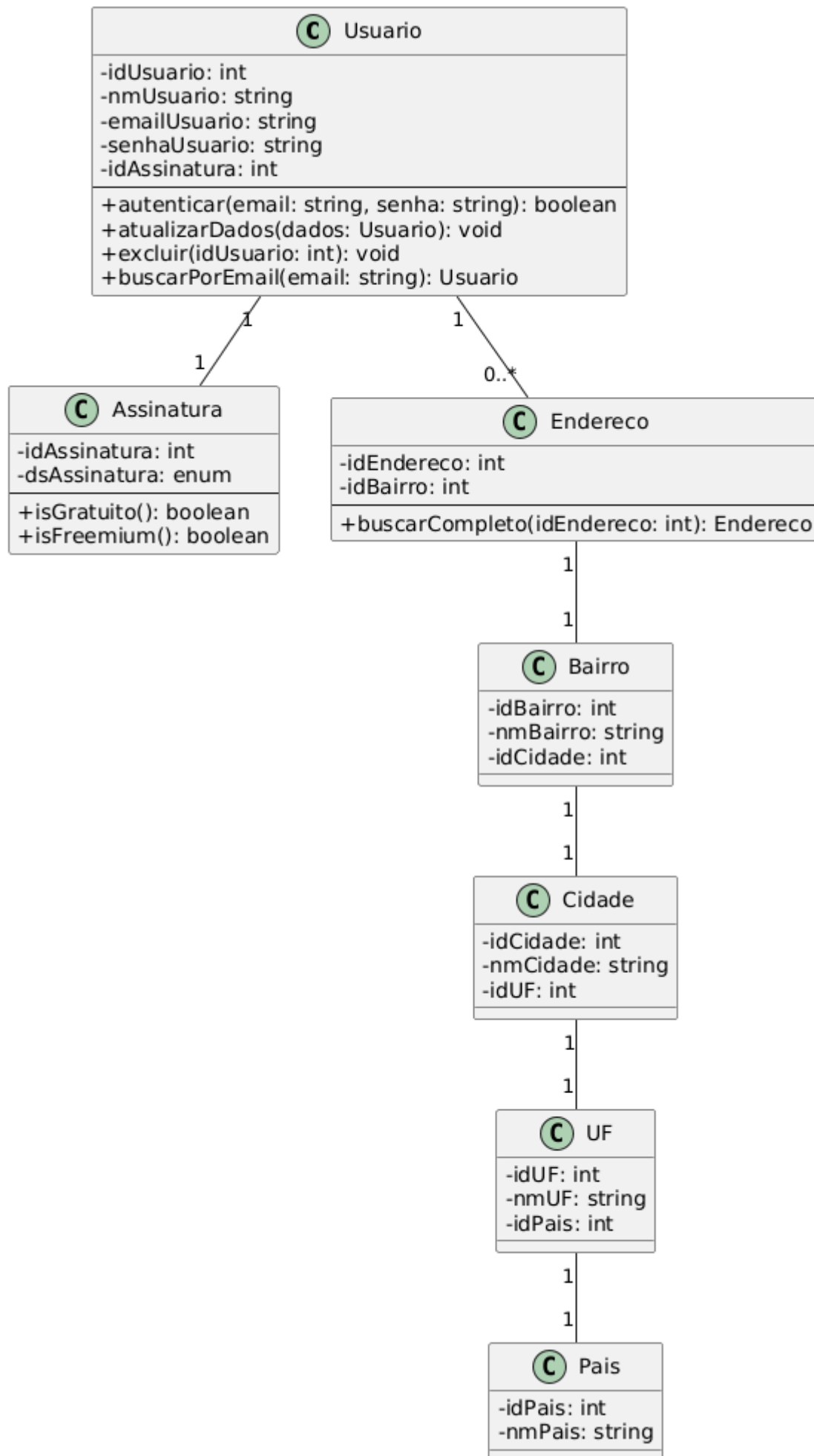
A tabela abaixo descreve cada pacote:

dominio.usuario	Contém as classes responsáveis pela identidade, autenticação e localização do usuário no sistema.
dominio.lista	Contém as classes que representam o núcleo funcional do sistema: listas de compras, itens e controle de gastos
dominio.comunidade	Contém as classes responsáveis pelo compartilhamento de listas entre múltiplos usuários
dominio.financieiro	Contém as classes de controle de despesas, rendas e geração de relatórios
dominio.notificacao	Contém a classe responsável pela representação e envio de notificações push
dominio.comentario	Contém as classes de avaliação e comentário de produtos, incluindo marca

1.3. Pacotes de Design Significativos do Ponto de Vista da Arquitetura

Pacote: dominio.usuario

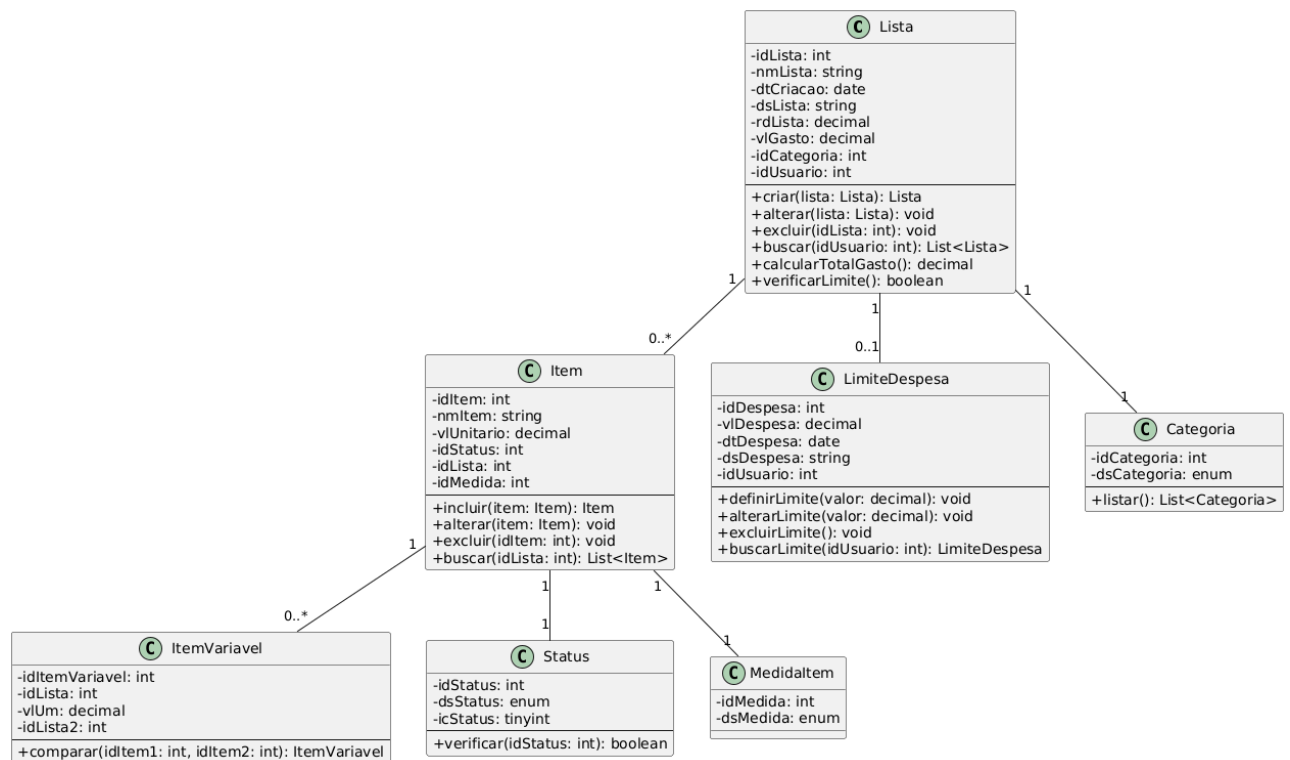
--	--



--	--

Usuario	Representa o usuário do sistema	Autenticação, atualização de dados, controle de assinatura
Assinatura	Representa o plano do usuário (Gratuito ou Freemium)	Determinar permissões de acesso às funcionalidades
Endereco	Representa o endereço vinculado ao usuário ou comunidade	Localização geográfica para busca de produtos
Bairro	Subdivisão do endereço	Referência geográfica
Cidade	Subdivisão do endereço	Referência geográfica
UF	Unidade federativa	Referência geográfica
País	País de origem	Referência geográfica

Pacote: dominio.lista

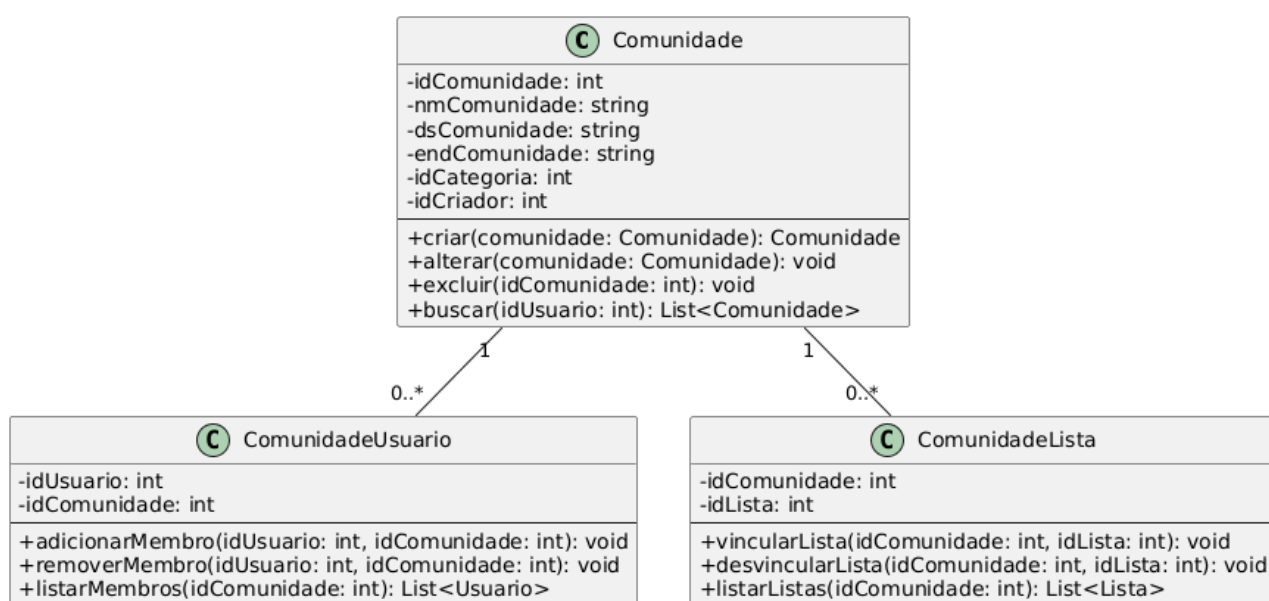


Lista	Representa uma lista de compras do usuário	Criação, edição, exclusão e cálculo de gastos totais
Item	Representa um produto dentro de uma lista	Inclusão, remoção e controle de quantidade
ItemVariavel	Representa variações de preço de um item para comparação	Comparação de preços entre lojas ou versões do produto

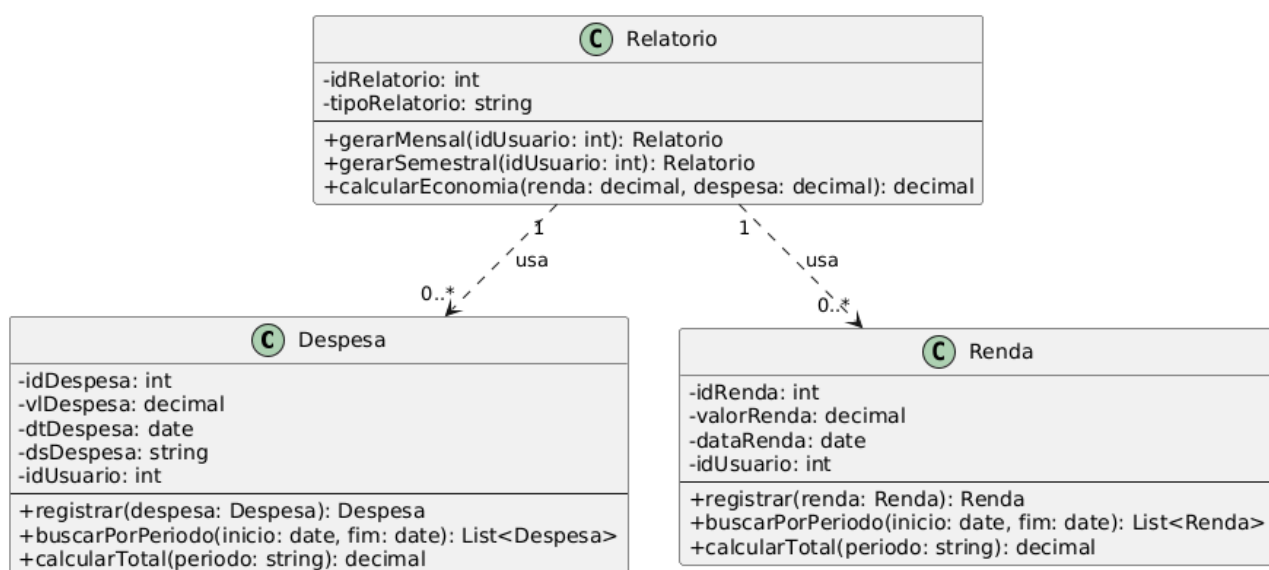
--	--

LimiteDespesa	Representa o limite de gasto definido pelo usuário para uma lista	Controle de orçamento por lista
Status	Representa o estado de um item (pendente, comprado, etc.)	Controle do ciclo de vida do item na lista
Categoria	Representa a categoria de uma lista ou produto	Classificação e filtro
MedidaItem	Representa a unidade de medida de um item	Padronização de quantidades

Pacote: dominio.comunidade



Pacote: dominio.financeiro

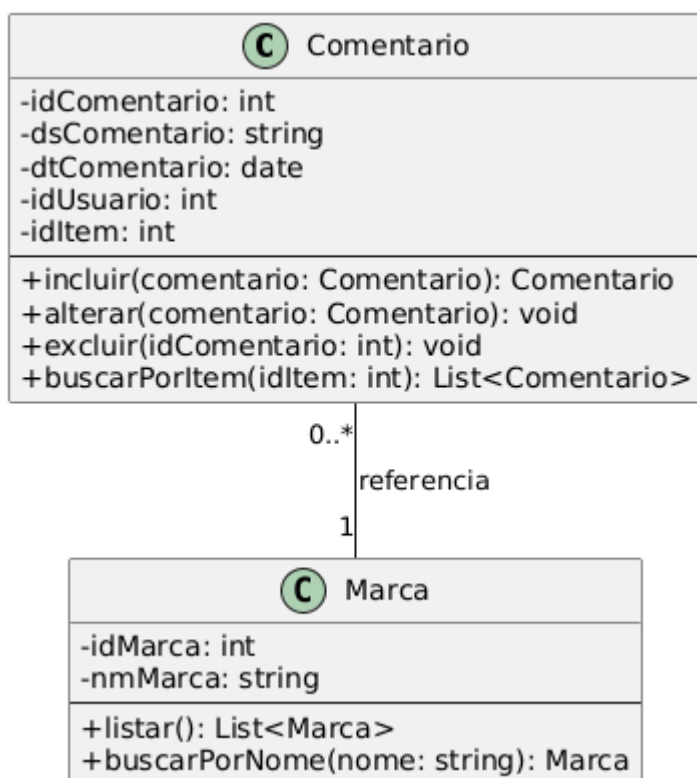


--	--

Comunidade	Representa um grupo de usuários que compartilham listas	Criação e administração de comunidades
ComunidadeUsuario	Associação entre comunidade e usuário	Controle de membros e permissões
ComunidadeLista	Associação entre comunidade e lista	Compartilhamento de listas dentro de uma comunidade

Despesa	Representa um gasto registrado pelo usuário	Registro e consulta de despesas por período
Renda	Representa uma entrada financeira do usuário	Registro e consulta de rendas por período
Relatorio	Agrega despesas e rendas para geração de relatórios gráficos	Geração de relatórios mensais e semestrais com cálculo de economia

Pacote: dominio.comentario



Comentario	Representa a avaliação de um usuário sobre um produto	Inclusão, edição e exclusão de avaliações
Marca	Representa a marca de um produto	Identificação e filtro de produtos por fabricante

--	--

Pacote: dominio.notificacao

 Notificacao
-idNotificacao: int -dsNotificacao: string -dtNotificacao: date -idUserario: int -idLista: int
+enviarLembreteLista(idLista: int, idUsuario: int): void +listarPendentes(idUsuario: int): List<Notificacao>

Notificacao	Representa uma notificação push enviada ao usuário	Envio de lembretes de listas pendentes via push notification
--------------------	--	--

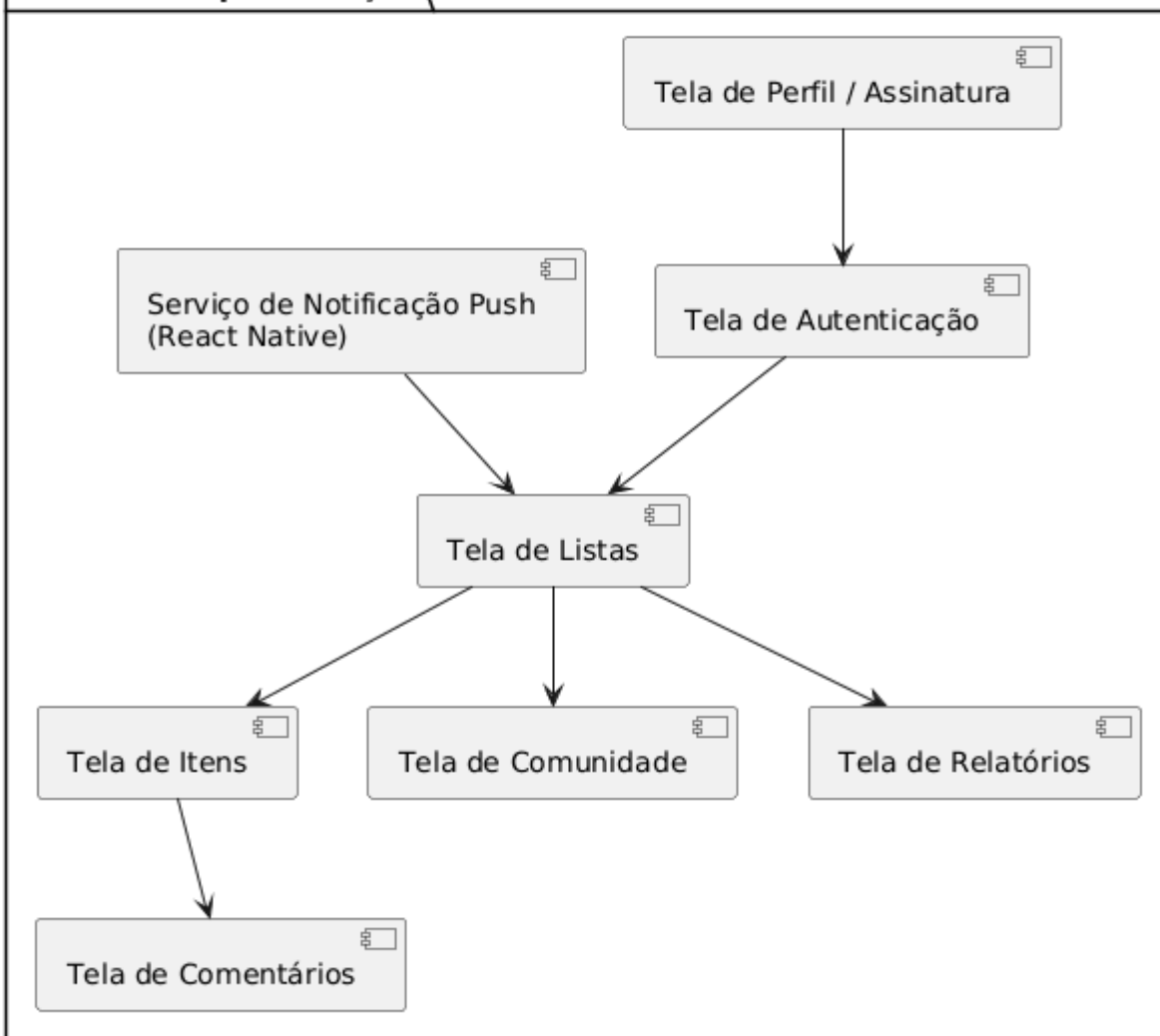
1.4. Camadas

O sistema é organizado em quatro camadas principais, descritas a seguir.

1.4.1. Camada de Apresentação (View)

Responsável pela interface com o usuário final. Implementada em React Native, consome a API REST exposta pelos controllers do backend.

Camada de Apresentação

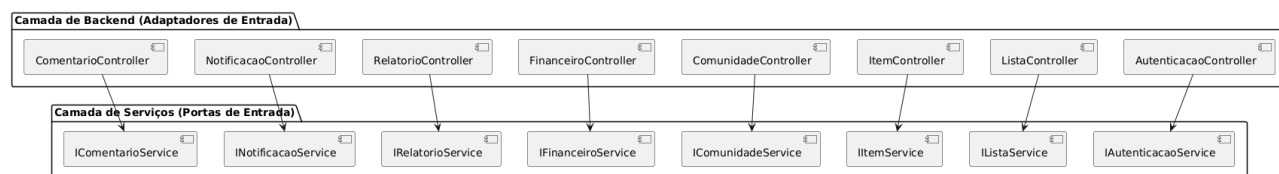


Tela de Autenticação	Login, cadastro e recuperação de senha.
Tela de Listas	Criação, edição e listagem de listas de compras.
Tela de Itens	Adição, edição e comparação de preços de itens.
Tela de Comunidade	Criação e gerenciamento de comunidades.
Tela de Relatórios	Visualização gráfica de gastos e economias.
Tela de Perfil/Assinatura	Gerenciamento do perfil e plano do usuário.
Tela de Comentários	Avaliação e comentários sobre produtos.
Serviço de Notificações Push	Recebimento e exibição de notificações push.

1.4.2. Camada de Backend (API)

Responsável por receber as requisições da camada de apresentação, acionar os serviços de domínio e retornar as respostas. Implementada em Node.js, expõe uma API REST.

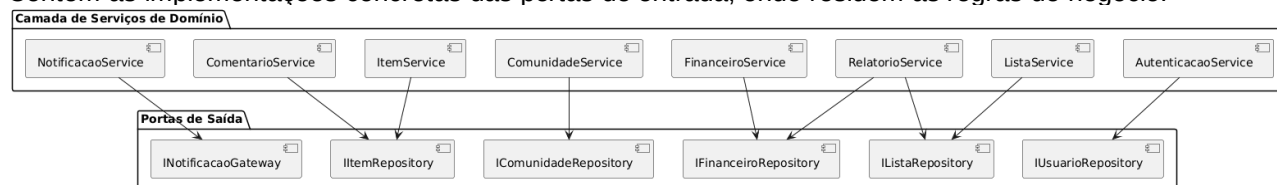
--	--



AutenticacaoController	Recebe requisições de login, cadastro e atualização de usuário
ListaController	Recebe requisições de CRUD de listas
ItemController	Recebe requisições de CRUD de itens e comparação de preços
ComunidadeController	Recebe requisições de CRUD de comunidades
FinanceiroController	Recebe requisições de registro de despesas, rendas e limites
RelatorioController	Recebe requisições de geração de relatórios
NotificacaoController	Recebe e despacha notificações push
ComentarioController	Recebe requisições de CRUD de comentários

1.4.3. Camada de Serviços de Domínio

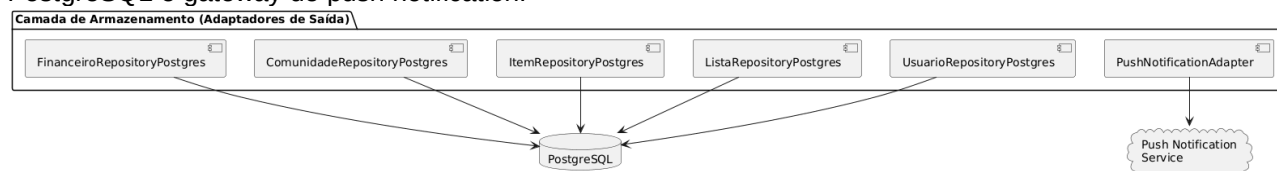
Contém as implementações concretas das portas de entrada, onde residem as regras de negócio.



1.4.4. Camada de Armazenament

o (Adaptadores de Saída)

Responsável pela persistência dos dados e integração com serviços externos. Implementada com PostgreSQL e gateway de push notification.



UsuarioRepositoryPostgres	Persistência e consulta de usuários, endereços e assinaturas
ListaRepositoryPostgres	Persistência e consulta de listas e seus metadados
ItemRepositoryPostgres	Persistência e consulta de itens, variações de preço e comentários

--	--

ComunidadeRepositoryPostgres	Persistência e consulta de comunidades e suas associações
FinanceiroRepositoryPostgres	Persistência e consulta de despesas, rendas e limites
PushNotificationAdapter	Integração com o serviço de push notification do React Native

7. Visão de Processos

Esta seção descreve a decomposição do sistema em processos leves (threads simples de controle) e processos pesados (agrupamentos de processos leves), Organize a seção em grupos de processos que se comunicam ou interagem. Descreva os modos principais de comunicação entre processos, como transmissão de mensagens e interrupções

Represente apenas dois processos, através de um diagrama de sequência e diagrama de atividades de um processo de negócio que o sistema deverá executar. (então são 4 diagramas)

8. Visão de Implantação

Esta seção descreve uma ou mais configurações da rede física (hardware) na qual o software é implantado e executado. Ela é uma visão do Modelo de Implantação. No mínimo, para cada configuração, ela deve indicar os nós físicos (computadores, CPUs) que executam o software e suas interconexões (barramento, e protocolos de comunicação) É incluído também um mapeamento dos processos da Visão de Processos nos nós físicos..

Diagrama de componentes físicos para implantação. Aqui apresentaremos o diagrama de implantação com os componentes (softwares), que estarão dentro de cada nó.

9. Visão da Implementação

Esta seção descreve a estrutura geral do modelo de implementação, a divisão do software em camadas e os subsistemas no modelo de implementação e todos os componentes significativos do ponto de vista da arquitetura, representados pelo diagrama de componentes. **Do ponto de vista da arquitetura que será utilizada.**

--	--

1.5. Visão Geral

[Esta subseção nomeia e define as diversas camadas e o seu conteúdo, as regras que determinam a inclusão em uma camada específica e as fronteiras entre as camadas. Inclua um diagrama de componentes que mostre os relacionamentos entre as camadas.]

Diagrama geral que demonstra a comunicação entre as camadas do projeto e os componentes principais. Este apresenta a arquitetura a ser utilizada, seus componentes e como eles se comunicam.

10. Visão de Dados (opcional)

[Uma descrição da perspectiva de armazenamento de dados persistentes do sistema. Esta seção será opcional se os dados persistentes forem poucos ou inexistentes ou se a conversão entre o Modelo de Design e o Modelo de Dados for trivial.]

11. Tamanho e Desempenho

[Uma descrição das principais características de dimensionamento do software que têm um impacto na arquitetura, bem como as restrições do desempenho desejado.]

12. Qualidade

[Uma descrição de como a arquitetura do software contribui para todos os recursos (exceto a funcionalidade) do sistema: extensibilidade, confiabilidade, portabilidade e assim por diante. Se essas características possuírem significado especial, como implicações de segurança, garantia ou privacidade, elas deverão ser delineadas claramente.]

13. Anexos

[Relacione aqui eventuais documentos que fazem parte do presente artefato, tais como atas de reunião, cronograma e outros.]

14. Referências

[Relacione aqui referências a documentos, sítios na Internet, manuais ou qualquer outro item que tenha sido usado para a confecção do presente artefato.]

--	--

15. Aprovações

Aprovações		
Participante	Assinatura	Data